



# LUPUSAI: MANUAL DE ARQUITECTURA Y DESPLIEGUE

*Guía Técnica de Configuración, Inicialización y Gestión de Entornos de Servidores Locales B2B*

Este documento constituye el manual técnico oficial para la puesta en marcha, gestión y administración del sistema **LupusAI** en entornos de producción local. El objetivo es estructurar de forma clara y reproducible los pasos necesarios para inicializar el backend asíncrono y enlazarlo operativamente con la interfaz clínica del usuario.

## 1. Arquitectura Organizacional de Directorios

Para garantizar el aislamiento de la lógica de negocio y facilitar un despliegue modular de grado comercial, el proyecto adopta una estructura jerárquica estricta. Cada componente ocupa un espacio unificado en el almacenamiento físico:

```
LupusAI_Project/
├── backend/
│   ├── main.py # Servidor de API Asíncrono (FastAPI)
│   ├── inference_core.py # Core del Motor de Inferencia y Red Convolucional
│   ├── requirements.txt # Manifiesto de Dependencias del Sistema
│   └── lupus_model.weights.h5 # Pesos Sinápticos Calibrados de la Red Neuronal
├── frontend/
│   └── index.html # Interfaz de Usuario Clínica y Panel de Triage
├── notebooks/
│   └── LupusAI_Model_Training.ipynb # Pipeline Completo de R&D y Pipeline de Datos
├── assets/
│   ├── grad_cam_sample.png # Mapas de Calor de Interpretabilidad Visual
│   └── confusion_matrix.png # Matriz de Validación del Diagnóstico Diferencial
└── README.md # Documentación Maestra del Repositorio
```

## 2. Prerrequisitos y Configuración del Entorno

En sistemas operativos Windows que utilizan administradores de entornos basados en Python o Miniconda, las variables de entorno pueden presentar restricciones de acceso global para ejecutables directos. Para subsanar este comportamiento de manera estandarizada y segura, se utiliza el lanzador nativo de Windows `py` como puente de ejecución de módulos de forma explícita.

**Regla de Oro del Administrador:** Siempre que se invoquen herramientas de empaquetado o servidores de producción, se antepondrá el comando `py -m`. Esto garantiza la correcta resolución de rutas binarias internas del sistema sin alterar las variables de entorno globales de la máquina.

### 3. Preparación de Archivos Base del Servidor

Antes de iniciar el servicio, verifique que los archivos clave contengan la configuración estructural adecuada para la comunicación entre capas (CORS habilitado en el Backend y endpoints apuntando a la dirección IP local de loopback desde el Frontend):

#### A. Especificación de Dependencias (backend/requirements.txt)

Asegure la existencia de las librerías base con sus respectivas restricciones de versiones:

```
tensorflow>=2.15.0
numpy
pillow
fastapi
uvicorn
python-multipart
```

#### B. Origen Cruzado y Endpoints (backend/main.py)

El backend debe incorporar el middleware `CORSMiddleware` configurado con `allow_origins=["*"]` para evitar bloqueos de seguridad del navegador al abrir el archivo HTML local del frontend.

### 4. Paso a Paso de Ejecución (Puesta en Marcha)

Siga estrictamente la siguiente secuencia de comandos en su consola de comandos (CMD de Windows) o la terminal integrada de Visual Studio Code para inicializar el sistema completo:

#### Paso 1: Posicionamiento de la Terminal

Abra la consola de comandos. Si se encuentra en la raíz del proyecto (LupusAI\_Project/), desplácese explícitamente hacia el directorio del servidor de backend. Esto es crucial para que los scripts localicen el archivo de pesos relativos:

```
C:\LupusAI_Project> cd backend
C:\LupusAI_Project ackend>
```

#### Paso 2: Instalación de Dependencias

Instale de forma masiva los paquetes requeridos por el núcleo de inferencia matemática utilizando el invocador de Python:

```
C:\LupusAI_Project ackend> py -m pip install -r requirements.txt
```

#### Paso 3: Inicialización del Servidor Uvicorn

Levante el servidor asíncrono ASGI apuntando a la aplicación `app` definida dentro de su archivo `main.py`. El parámetro `--reload` permite al servidor reiniciar automáticamente si realiza modificaciones en el backend:

```
C:\LupusAI_Project ackend> py -m uvicorn main:app --reload
```

#### Paso 4: Verificación de Estado en Consola

El arranque exitoso del microservicio clínico se confirma cuando la consola imprima las siguientes líneas informativas:

```
INFO: Will watch for changes in these directories...
INFO: Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
⌚ Cargando red neuronal en memoria... (Esto puede tomar unos segundos)
✅ Motor de IA cargado y listo para inferencia.
```

#### Paso 5: Activación de la Interfaz del Médico

Con la consola activa en segundo plano (no cierre la ventana):

1. Abra el explorador de archivos de Windows y navegue a `LupusAI_Project/frontend/`.
2. Haga doble clic sobre el archivo **index.html** para inicializar la interfaz clínica en su navegador web.
3. Cargue una imagen de prueba; la interfaz transmitirá los datos binarios al puerto 8000 y renderizará el reporte con la alerta diagnóstica correspondiente de manera inmediata.

## 5. Ciclo de Vida del Servidor (Gestión y Apagado)

Para administrar adecuadamente el consumo de recursos de hardware de la computadora, se deben seguir protocolos de cierre controlados:

| Acción Clínica/<br>Técnica         | Comando /<br>Combinación                           | Efecto Operativo                                                                                         |
|------------------------------------|----------------------------------------------------|----------------------------------------------------------------------------------------------------------|
| <b>Pausar / Apagar Servidor</b>    | <b>Ctrl + C</b> (Dentro de la Consola)             | Envía un flag de interrupción segura (SIGINT), libera los puertos de red ocupados y descarga la GPU/RAM. |
| <b>Cierre Total de Terminal</b>    | <b>exit</b> (O clic en la "X" de ventana)          | Termina de manera definitiva el proceso del intérprete de comandos en el sistema operativo.              |
| <b>Consultar Documentación API</b> | Navegar a: <code>http://127.0.0.1:8000/docs</code> | Abre la interfaz Swagger automatizada para probar peticiones HTTP estructuradas.                         |

**Resolución de Conflictos Comunes (Error de Puerto Ocupado):** Si el servidor se apaga abruptamente sin presionar **Ctrl + C**, el puerto 8000 podría quedar bloqueado por un proceso huérfano. Para solucionarlo sin reiniciar el equipo, cierre la terminal por completo, abra una nueva ventana y ejecute el comando de encendido nuevamente; el cargador de Uvicorn reasignará los listeners de forma automática.